Experimental Results & Interpretation

Experimental Results & Interpretation: LLM Code Environmental Impact

Dataset Analyzed: `median_emissions.csv`
Scope: 21 Algorithms across 4 Categories (Dynamic Programming, Graph, Matrix, Sorting)
Evaluated LLMs: ChatGPT, Claude, Gemini
Total Benchmarks: 63 unique (Algorithm, Scale) evaluations

EXECUTIVE SUMMARY

Across 63 benchmark evaluations, the carbon emissions (*kg* CO_2*eq*) of code generated by ChatGPT, Claude, and Gemini demonstrate significant performance and environmental differences:

LLM Model	Lowest Emissions Wins	Win Rate (%)	Category Dominance	Architectural Profile
Claude	**26 / 63**	**41.3%**	Graph (66.7%), Sorting (46.7%)	Employs C-optimized Python idioms (`zip(*matrix)` , `heapq`). Highest stability without tail-latency spikes.
Gemini	**25 / 63**	**39.7%**	Dynamic Programming (73.3%), Matrix (46.7%)	Generates minimal table-filling loops for DP, but susceptible to dynamic allocation spikes (`.append()`) at large scales.

LLM Model	Lowest Emissions Wins	Win Rate (%)	Category Dominance	Architectural Profile
ChatGPT	**12 / 63**	**19.0%**	Basic Sorting (Merge Sort)	Focuses on high readability, but relies on object extensions (`list.extend`) causing memory and emission bloat.

🔍 CATEGORY-BY-CATEGORY ANALYSIS

1. Dynamic Programming (5 Algorithms, 15 Benchmarks)

- **Winner**: **Gemini** (11 / 15 category wins)
- **Mechanics**: Dynamic programming performance is dictated by memory access patterns and table lookup efficiency. Gemini consistently generated minimalist table-filling loops with minimal object allocation overhead.
- **Key Algorithms**: `edit_distance` , `fibonacci_memoized` , `knapsack_01` , `lcs` , `optimal_bst` .

2. Graph Algorithms (6 Algorithms, 18 Benchmarks)

- **Winner**: **Claude** (12 / 18 category wins)
- **Mechanics**: Graph algorithms depend on data structure selection (adjacency lists, priority queues). Claude leveraged standard C-optimized library constructs (`heapq`), outperforming custom array-scanning alternatives.
- **Key Case Study (`prim` at Scale 1000)**:
  - Claude (`heapq` min-heap): **5.07×10^{-7} kg CO_2eq**
  - ChatGPT & Gemini (array scanning): **1.66×10^{-6} kg CO_2eq** (**3.3× higher emissions**)

3. Matrix Operations (5 Algorithms, 15 Benchmarks)

- **Winner**: **Gemini** (7 / 15 wins), followed by **Claude** (5 / 15 wins)
- **Mechanics**: Matrix operations show the largest divergence between models due to $O(N^3)$ computational scaling.
- **Severe Discrepancies**:
  - `matrix_transpose` **at Scale 1024**:

- Claude: `[list(row) for row in zip(*matrix)]` (Built-in C iterator) → **5.83×10^{-9} kg CO_2eq**
- Gemini: Dynamic `.append()` in nested loops for 1,048,576 elements → **4.21×10^{-7} kg CO_2eq** (**72.7× higher emissions**)

- `matrix_determinant` **at Scale 128**:
  - Gemini: Unbounded Euclidean division loop → **7.04×10^{-7} kg CO_2eq** (**122.7× higher emissions** than Claude at 5.73×10^{-9}).

4. Sorting Algorithms (5 Algorithms, 15 Benchmarks)

- **Winner**: **Claude** (7 / 15 wins), followed by **ChatGPT** (5 / 15 wins)
- **Mechanics**: Standard $O(N \log N)$ comparison sorts yield comparable emissions across models at scale 100,000 ($\approx 4.4 \times 10^{-7}$ kg CO_2eq).
- **Anomalies**:
  - In `counting_sort` at Scale 1,000 and 10,000, ChatGPT used repeated `list.extend()` calls, incurring **34× to 41× higher emissions** than Claude's index placement.

ROOT-CAUSE ANALYSIS: EMISSION DRIVERS IN GENERATED CODE

1. **Python C-Extension Utilization**: Using Python's C-implemented built-ins (`zip` , `heapq` , slice copies) reduces CPU cycle counts and garbage collection overhead, lowering carbon emissions by up to **72×**.
2. **Dynamic Memory Reallocation**: Appending elements (`list.append()`) within deeply nested $O(N^2)$ or $O(N^3)$ loops destroys spatial locality and triggers frequent memory re-allocations.
3. **Algorithm Variant Selection**: Unconstrained reduction loops without numerical tolerances cause extreme CPU time spikes, translating directly into carbon emissions.

RECOMMENDATIONS FOR GREEN SOFTWARE DEVELOPMENT

1. **Model Choice**:
   - **Claude** is the most reliable model for general algorithmic tasks, avoiding severe worst-case emission spikes.
   - **Gemini** is optimal for tight array/DP algorithms, provided dynamic list expansion is avoided.

2. **Prompting Strategy**: Explicitly prompt LLMs to use pre-allocated arrays, C-extension built-ins, and vectorization to minimize software carbon footprint.